

FAME: EVALUATING LANGUAGE MODELS USING FALSE MATHEMATICAL EQUATIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

This paper presents the Falsely Altered Mathematical Evaluation (FAME), a benchmark designed to study how Large Language Models (LLMs) handle logical reasoning when exposed to false mathematical facts. By introducing fabricated equations (e.g., “ $2 + 2 = 5$ ”) and testing performance across multi-hop expressions of varying difficulty, FAME measures four key axes: adherence to false facts, degradation under added facts, difficulty sensitivity, and performance deterioration. We evaluate state-of-the-art models alongside a scaling study of the LLaMA family. Results show that high-capacity models achieve strong adherence when evaluating expressions (above 92%), but performance declines on algebraic tasks, dropping above 60%. A clear scaling trend emerges: larger models are more stable, suggesting scale is central to generalizing altered rules.

1 INTRODUCTION

Although LLMs have demonstrated great capabilities in retrieving and manipulating factual information (?), such as geopolitical data (?) (?) and scientific knowledge (?) (?), their ability to handle mathematical reasoning remains different from textual understanding (?) (?) (?). World knowledge typically tolerates contextual variation (e.g., “the capital of France is Paris”), whereas mathematical rules rely on strict logical consistency, where a mere minor alteration of a single operation (e.g., changing “ $2 + 2 = 4$ ” to “ $2 + 2 = 5$ ”) can undermine the integrity of subsequent logical steps. Accordingly, mathematics serves as a rigorous benchmark for evaluating the internal consistency of model reasoning across knowledge editing, operation fidelity, and multi-hop reasoning processes.

This study is designed to evaluate the behavior of language models when dealing with faulty reasoning when presented with specific prompts. Benchmarks such as MQuAKE (?) and MRKE (?) assess multi-hop reasoning post-edit, but do not target structured numerical reasoning or arithmetic operations, leaving a critical gap. By measuring the extent to which models can adapt to counterfactual prompts, it becomes possible to assess and, to some degree, quantify their adaptability. Mathematics has been selected as the primary scope of inquiry because the generation of both problems and solutions in this domain lends itself to a modular and streamlined structure, allowing for deeper analysis in the future.

Therefore, we introduce a novel benchmark, FAME, designed to study knowledge editing using false mathematical equations, specifically core PEMDAS operations. Our approach evaluates how deterministic rules (e.g., changing “ $2+2=4$ ” to “ $2+2=5$ ”) propagate through multi-step calculations, an area where current model editing techniques and benchmarks are nonexistent (?) (?) (?) (?). We’ve released the benchmark publicly at <https://anonymous.4open.science/r/fame-7183/README.md>.

2 RELATED WORKS

2.1 BENCHMARKS FOR FACTUAL EDITING

As more research has been done on knowledge editing, several benchmarks, such as CounterFact (?) and ZsRE (?), have emerged to evaluate the fidelity, generalization, and side effects of model edits. Although they are useful for tracking local and global edit success in textual knowledge, they

do not assess how edits influence logical computation. Other recent benchmarks, such as MQuAKE (?), PokeMQA (?), and MRKE (?) extend evaluation to multi-hop reasoning questions, tracking consistency, and error propagation. RippleEdits (?) further contributes by quantifying how factual changes ripple downstream in model outputs.

Despite advancements, these benchmarks still treat knowledge as static, limiting their ability to capture domains like mathematics. In such fields, a simple false edit like changing " $2 + 2 = 4$ " to " $2 + 2 = 5$ " can disrupt entire reasoning pipelines, unlike factual edits (e.g., "Paris is the capital of France"), which remain localized.

2.2 LANGUAGE MODELS AND MATHEMATICAL REASONING

While modern LLMs demonstrate impressive mathematical problem-solving abilities (?), they remain held back in their capacity to manipulate procedural mathematical knowledge. Contemporary solutions come in three approaches, each with significant limitations:

1. **Editing Techniques:** Traditional knowledge editing methods, which often involve modifying model weights, successfully change declarative facts but generally do not challenge the formulated nature of mathematical thought. As shown in ?, these techniques struggle with logical consistency when updates involve dependent rules. Prompt-based methods, while more flexible, can also face challenges when factual prompts conflict with existing model knowledge (?).
2. **Reasoning Benchmarks:** Existing multi-hop QA benchmarks, such as MQuAKE (?) and MRKE (?), focus on textual inference rather than logical operations (??). Furthermore, while work like ? reveals the mathematical brittleness of LLMs, it does not investigate how edited rules propagate through computations, a gap our work addresses.
3. **Procedural Understanding:** Diagnostic studies (?) (?) show LLMs are inclined to rote-memorize arithmetic patterns rather than acquire generalizable rules (?), and they fail under logical perturbations (operand swaps (?), etc.) while also exhibiting flawed reasoning when basic axioms are falsely adjusted (?).

Most importantly, while works like MathPrompter (?) improve math performance by fine-tuning, no existing technique enables editable procedural knowledge in LLMs, where rules of mathematics can be updated cleanly without forgetting catastrophically (?) or being inconsistent (?). To bridge this gap, our benchmark relies on prompting to see how well models directly interpret these edited facts, allowing us to observe the model’s adaptability and ability to change their inner thinking.

3 DATASET

Falsely Altered Mathematical Evaluation (FAME) is a benchmark designed to evaluate LLMs’ ability to evaluate mathematical edits in multi-hop reasoning processes, allowing us to analyze how a single or multiple points of misinformation can affect the model’s performance. Problems in this benchmark can be one of two kinds: evaluating an expression or the value of an unknown variable in an equation.

Our methodology for the generation and evaluation of FAME consists of a three-stage pipeline: (1) Initial Fact Injection, (2) Expression Generation, and (3) Evaluation Framework. Each component is designed to evaluate the model along four parts: false fact adherence, false fact degradation, difficulty degradation, and performance deterioration.

3.1 INITIAL FACT INJECTION

We begin by selecting a base set of incorrect arithmetic equations using the operations of PEMDAS. For example, " $2 + 2 = 5$," " $3 / 3 = 2$," and " $2 ** 3 = 10$ " (** meaning exponents). Every fact that is required for a problem is then added to a list that will be directly prompt-fed to models, altering their original adherence to conventional math.

For the two different kinds of problems, expression-evaluation and algebra problem-solving, the same amount of base false facts are initialized. For problems involving evaluating the value of a

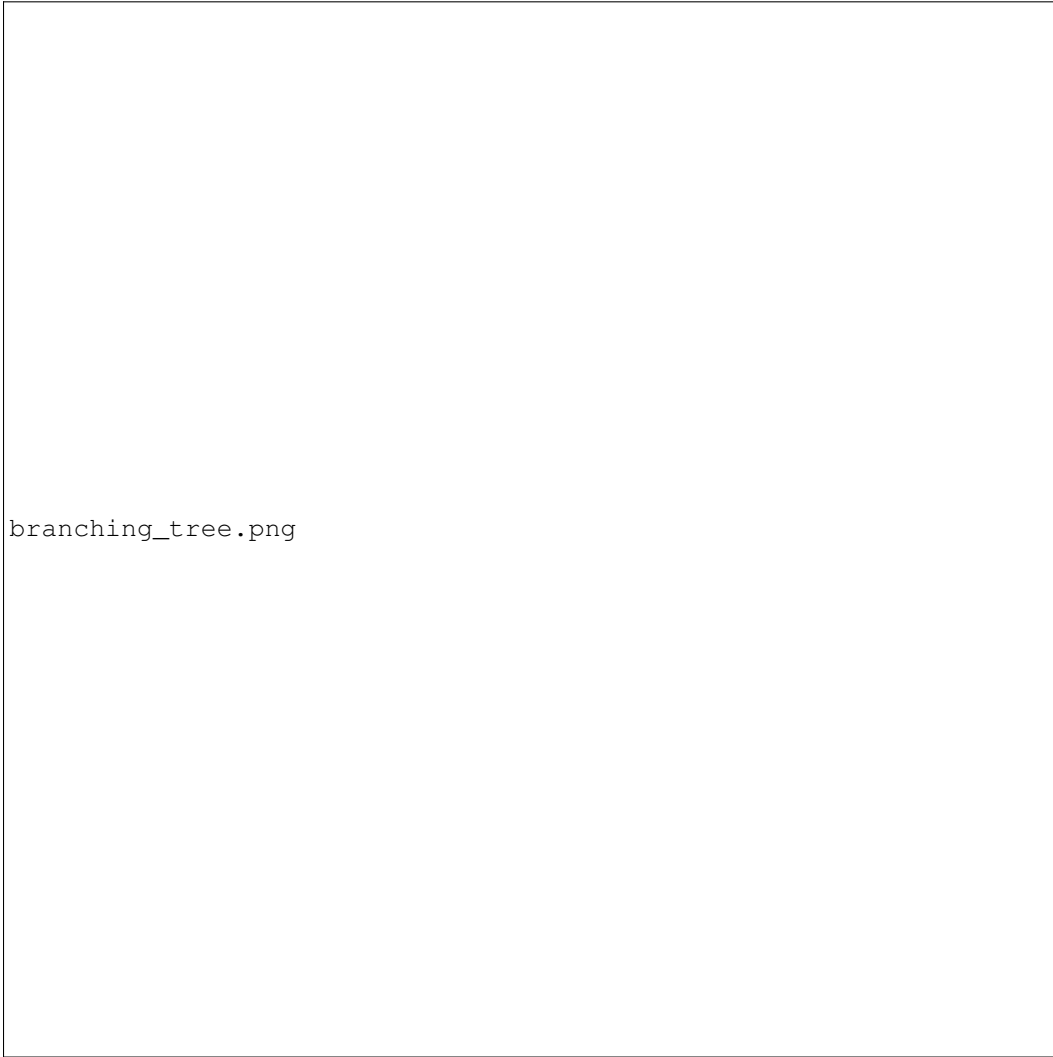


Figure 1: The modification structure of FAME. A single false mathematical expression may undergo three different modifications: (1) adding an operation, (2) layering an expression, and (3) layering a false mathematical fact.

single expression, a random number of one to three false facts can be initialized for the problem. For problems involving algebra, there are two sides to equations; only the left-hand side is initialized with base facts, so one to three random facts are generated solely to be added to the left-hand side.

3.2 EXPRESSION GENERATION

Following the collection of false mathematical facts, we then generate expressions to evaluate the model’s ability to reason over edited mathematical knowledge. To explore model performance depth, we adopt a branching strategy, where each false math equation is transformed by different modifications that make the expression more complicated to evaluate as modifications are added. This branching method allows for the existence of problems ranging from minimal problem-solving to precise calculations and accuracy.

Each false mathematical equation serves as the root of a branching structure, with transformations applied to increase the complexity of problems as seen in Figure 1. Transformations for problems can be one of the following three:

1. One of the four basic operations and a number is added to the expression, encasing the previous expression in parentheses, e.g. "+ 3" is added to "2 + 2" to make "(2+2) + 3"
2. A two-number expression using one of the four basic operations is inserted into the existing expression replacing a previous number which had the same value, e.g. "(2+2)" is inserted into "4 + 3" to make "(2+2) + 3"
3. A false mathematical fact is, like the second transformation, inserted into the existing expression, and the new fact is added to the list of false facts to be prompted to the model, e.g. "2 + 2 = 5" is the false fact and is inserted into "5 + 3" to make "(2+2) + 3" while at the same time the false fact "2 + 2 = 5" is added to the list of false facts

To ensure both complexity and variety of difficulty, there is a set amount of transformations for all problems. For expression-evaluation, a random number of one to three transformations will be added to each problem. For algebra problem-solving, like expression-evaluation, a random number of one to three transformations will be added to the left-hand side of the equation, and the right side can have one to five transformations.

Difficulty is counted by totaling the sum of the number of operators in the expression or equation. For expression-evaluation, the difficulty ranges from 2 to 8. For algebra problem-solving, since there are two sides of the equation, the difficulty ranges from 3 to 11. The variety of randomly generated equations allows us to vary both the depth and difficulty of problems, allowing metrics to calculate model performance as more and more modifications are added.

3.3 EVALUATION FRAMEWORK

We proceed to evaluate the model's performance under the assumption that the edited fact is true. To ensure the model's understanding, each expression is presented to the model in a prompt (shown in Appendix A) that introduces the task, provides examples, and states formatting constraints on the output.

Each model answer is then categorized as one of the following: edit-aligned if it matches the answer with the false mathematical equation in mind, real-aligned if it matches the arithmetically correct result, and incorrect if it matches neither. The percentage of answers that are edit-aligned is classified as the Edited-Aligned Percentage (EAP); the percentage of answers that are real-aligned is classified as the Real-Aligned Percentage (RAP); the percentage of answers that are incorrect is classified as the Incorrect Answer Percentage (IAP).

$$EAP = \frac{\text{Edited-Aligned Answers}}{\text{Total Answers}} \times 100$$

$$RAP = \frac{\text{Real-Aligned Answers}}{\text{Total Answers}} \times 100$$

$$IAP = \frac{\text{Incorrect Answers}}{\text{Total Answers}} \times 100$$

Control expressions based on real-world mathematics are also evaluated. The percentage of outputs that match the arithmetically correct result is referred to as the Arithmetically Correct Answer Percentage (ACAP). The percentage of outputs that fail to match the correct result is defined as the Arithmetically Incorrect Answer Percentage (AIAP).

$$ACAP = \frac{\text{Arithmetically Correct Answers}}{\text{Total Answers}} \times 100$$

$$AIAP = \frac{\text{Arithmetically Incorrect Answers}}{\text{Total Answers}} \times 100$$

This classification allows us to compute performance metrics along the four major axes:

1. False Fact Adherence (FFA) measures the extent to which a model adheres to a false mathematical equation across multi-hop expressions. This metric is computed as:

$$\text{FFA} = \frac{\text{EAP}}{\text{EAP} + \text{RAP}}$$

This formulation focuses only on cases where the model demonstrates consistent behavior, whether aligned with the false mathematical equation or with real-world arithmetic. By comparing the rate of EAP to the rate of RAP, the metric isolates the model’s alignment with the false equation. A high value shows great adherence to the false mathematical equation, while a low value suggests an increased chance of the model’s calculations using conventional mathematics.

2. False Fact Degradation (FFD) gives a number for the extent to which the model maintains its performance when an increasing number of false facts are added to a problem. The metric (with NF standing for the number of false math facts, EXP standing for the FAME expression-evaluation benchmark, and ALG standing for the FAME algebra-problem-solving benchmark) is computed as:

$$\text{FFD}_{EXP} = \frac{1}{5} \sum_{i=1}^5 (\text{EAP}_{\text{NF}_i} - \text{EAP}_{\text{NF}_{i+1}})$$

$$\text{FFD}_{ALG} = \frac{1}{10} \sum_{i=1}^{10} (\text{EAP}_{\text{NF}_i} - \text{EAP}_{\text{NF}_{i+1}})$$

The False Fact Degradation metric is different between expression and algebra problems as the two have different ranges of number of false facts per problem: one to six false facts per problem for expression problems and from one to eleven for algebra problems. The average of the subtraction of EAP values of problems with successive numbers of false facts estimates the rate of change in accuracy as the number of false facts increases. A low or even negative FFD indicates consistent or improving performance, while a high FFD indicates decreasing consistency in accuracy as problems contain more false facts.

3. Difficulty Degradation (DD) shows how a model’s arithmetic accuracy degrades as expression difficulty increases by measuring the average drop/slope in EAP between difficulty tiers. The metric (with DIFF standing for difficulty, EXP standing for the FAME expression-evaluation benchmark, and ALG standing for the FAME algebra-problem-solving benchmark) is computed as:

$$\text{DD}_{EXP} = \frac{1}{6} \sum_{i=2}^7 (\text{EAP}_{\text{DIFF}_i} - \text{EAP}_{\text{DIFF}_{i+1}})$$

$$\text{DD}_{ALG} = \frac{1}{8} \sum_{i=3}^{10} (\text{EAP}_{\text{DIFF}_i} - \text{EAP}_{\text{DIFF}_{i+1}})$$

The Difficulty Degradation metric is different between expression and algebra problems as the two differ in range of difficulty. The average of the subtraction of EAP values between successive difficulty levels provides an estimate of the average rate of change in accuracy as difficulty increases. A low DD shows stable performance across difficulty levels, while a high DD signals a large drop-off under increased complexity.

4. Performance Deterioration (PD) quantifies the drop-off in model performance due to the integration of a false mathematical equation, reflecting the extent to which the false mathematical equation integration causes broad model corruption in reasoning. This metric is computed as:

$$\text{PD} = \text{IAP} - \text{AIAP}$$

The calculation quantifies the increase in IAP when transitioning from real-world arithmetic to the edited false mathematical equation world. High PD signals major arithmetic disruption from the edit, while low PD indicates stable reasoning despite the change.

All metrics are reported across the entire dataset to enable insights into model behavior in multiple aspects.

Metric	FAME Expression Benchmark				
	GPT 4.1	Claude 3.5 Sonnet	Gemini 2.5 Pro	LLaMA 4 Maverick	Qwen 3 235B
EAP (%)	88.36	95.54	99.44	97.56	98.22
RAP (%)	0.22	0.04	0.06	0.14	0.04
IAP (%)	11.42	4.42	0.50	2.30	1.74
ACAP (%)	91.78	99.00	99.56	99.26	99.50
AIAP (%)	8.20	1.00	0.44	0.74	0.50
FFA (%)	99.75	99.96	99.94	99.86	99.96
FFD (%)	4.36	3.81	1.24	3.80	1.11
DD (%)	1.42	2.15	0.21	1.23	0.66
PD (%)	3.22	3.42	0.06	1.56	1.24

Table 1: Performance of state-of-the-art models on the FAME Expression benchmark, all values in percentages.

Metric	FAME Algebra Benchmark				
	GPT 4.1	Claude 3.5 Sonnet	Gemini 2.5 Pro	LLaMA 4 Maverick	Qwen 3 235B
EAP (%)	16.11	23.49	66.88	29.23	50.06
RAP (%)	4.36	2.10	0.56	2.68	0.71
IAP (%)	79.53	74.41	32.56	68.09	49.22
ACAP (%)	97.90	92.64	98.86	98.00	99.23
AIAP (%)	2.10	7.34	1.14	2.00	0.77
FFA (%)	78.69	91.79	99.17	91.60	98.60
FFD (%)	3.95	5.69	9.68	6.26	2.89
DD (%)	3.47	6.08	6.28	6.87	1.19
PD (%)	77.42	67.07	31.42	1.31	48.45

Table 2: Performance of state-of-the-art models on the FAME Algebra benchmark, all values in percentages.

4 MODEL EXPERIMENTATION

4.1 OVERVIEW OF EVALUATION SETUP

To evaluate the model under factual changes, we selected a group of five state-of-the-art LLMs, comprising of GPT-4.1 (OpenAI), Claude 3.5 Sonnet (Anthropic), Gemini 2.5 Pro (Google), LLaMA 4 Maverick, and Qwen-2.5-7B-Instruct-Turbo. Proprietary models were accessed via official APIs, while open models were run through the together.ai inference endpoint. The prompt instructs the model to assume an incorrect fact as true, following it up with an arithmetic expression, and return only the final numerical result without equations, explanation, punctuation, or formatting.

In order to extract the intended model response, generated responses were parsed using logic to get the proper numeric values. In cases where outputs were problematic, the expression was marked as skipped and excluded from analysis. This evaluation setup establishes a controlled framework for measuring adherence to false mathematical equations, logical fidelity, and reasoning depth under explicit factual edits.

Along with our state-of-the-art experimentation, we wanted to investigate how accuracy changes as model sizes increase. We introduce an experiment that directly compares performance across a family of models varying in parameter size but sharing the same architectural lineage. This controlled comparison may help identify whether specific trends—such as EAP robustness or susceptibility to logical drift—scale with model size.

The four models selected for this experiment all belong to the LLaMA family. At the smallest end is meta-llama/Llama-3.2-3B-Instruct-Turbo, followed by meta-llama/Meta-Llama-3.1-8B-Instruct-Turbo, then meta-llama/Meta-Llama-3.1-70B-Instruct-Turbo, and finally the largest, meta-

Metric	FAME Expression Benchmark				FAME Algebra Benchmark			
	3.2-3B	3.1-8B	3.1-70B	3.1-405B	3.2-3B	3.1-8B	3.1-70B	3.1-405B
EAP (%)	7.88	68.29	88.65	92.06	1.03	7.30	21.26	27.03
RAP (%)	2.27	1.45	0.21	0.24	1.37	4.65	1.58	1.47
IAP (%)	89.85	30.27	11.14	7.70	97.59	88.05	77.16	71.50
ACAP (%)	9.94	91.00	98.16	98.86	41.17	70.12	90.14	91.58
AIAP (%)	89.76	8.98	1.84	1.14	58.59	29.82	9.86	8.42
FFA (%)	77.63	97.93	99.77	99.74	42.98	61.11	93.09	94.85
FFD (%)	0.79	9.17	0.22	4.59	0.10	2.70	5.08	6.16
DD (%)	4.03	3.23	3.63	3.25	0.00	2.1	5.93	5.52
PD (%)	0.09	21.29	9.30	6.56	39.00	58.24	67.30	63.08

Table 3: Performance of different parameter-sized Llama models on the FAME Expression benchmark on the left side of the table and the FAME algebra benchmark on the right side of the table, all values in percentages.

llama/Meta-Llama-3.1-405B-Instruct-Turbo. Each successive model represents a significant increase in parameter count, providing an opportunity to isolate the effect of scale on fact-edit alignment performance and potentially uncover new scaling-related trends.

4.2 STATE-OF-THE-ART MODEL PERFORMANCE TRENDS

In the expression benchmark, all state-of-the-art models show great adherence to the false equation, with EAP above 88% and FFA above 99% in all models. This shows good alignment with the introduced fact even as the expression difficulty increases. Degradation remains small across these models, where FFD and DD stay under 4.5%, suggesting stable performance as problems become more complicated. PD values also stay low (<3.5%), showing little change from the integration of the false fact. Exact results shown in Table 1.

The Algebra benchmark introduces clear separation. All models have a major drop off in performance, with Gemini having the highest EAP (66.88%) and GPT having the lowest EAP (16.11%). Gemini and Qwen have high adherence (FFA 99.2% and 98.60%), but Gemini has moderate degradation (FFD 9.68%, DD 6.28%) while Qwen has low degradation (FFD 2.89%, DD 1.19%). Claude and GPT drop substantially in adherence (FFA 91.8% and 78.7%) and suffer steep performance deterioration, with PD values of 67% and 77%, respectively, showing struggle under algebraic complexity. Exact results shown in Table 2.

4.3 TRENDS FOR DIFFERENT MODEL SIZES

Across the Expression benchmark, model size strongly correlates with performance, with performance increasing as model size scales larger. Accuracy metrics show this effect: the 3B model’s Expression EAP is 7.88%, but the subsequent Expression EAPs rise in accuracy up to 92.06% for the 405B model. However, degradation metrics seem to remain very similar across models; DD varied very little between each model, and PD had a wide range of percentages but did not seem to have a correlation.

On the Algebra benchmark, the separation is also clear. The 3B model reports an EAP of 1.03% while the 405B model reports 27.03%, with models in between scaling in accuracy as well. Similar to the Expression benchmark, degradation metrics had very similar values to each other, with the DD for 3B even being 0%. However, this can likely be attributed to the fact that 3B’s accuracy was consistently very low across all difficulties. The results here suggest that increasing model size has a positive correlation with model accuracy, but consistency across various difficulties remains similar between models. Exact results shown in Table 3.

5 LIMITATIONS

Our current Falsely Altered Mathematical Evaluation (FAME) is limited to PEMDAS operations in expressions and algebra, providing a setting but not testing how models handle corruption in advanced mathematics such as calculus or discrete math, where different structures may give different outcomes. In addition, reasoning with altered facts may vary across other subjects like history, physics, or biology, which have distinct knowledge structures and reasoning patterns. Expanding to these domains would offer a greater view of how LLMs keep consistency under factual corruption. Furthermore, the FAME currently features a maximum of eight levels of difficulty for the Expression benchmark and eleven levels of difficulty for the Algebra benchmark through our recursive generation. While it is enough to demonstrate basic degradation trends, expanding the benchmark to include more difficulty levels could provide more insight on model capability under factual corruption. Future iterations of FAME could also explore a broader range of false fact types, investigate the impact of prompt engineering variations, and directly integrate with and evaluate various knowledge-editing algorithms.

6 CONCLUSION

This work introduces FAME, a novel benchmark designed to evaluate LLMs’ logical reasoning when confronted with false mathematical equations. Our findings demonstrate that larger parameter models are able to apply edited equations with consistency, achieving over 99% adherence on expression- problems and maintaining great false fact alignment. However, this stability declines when tested on algebraic benchmarks. Here, the largest models suffer performance drops exceeding 60%. A trend also emerges from scaling experimentation, which is that larger models show stronger adherence and accuracy across difficulty levels, while smaller models collapse almost entirely. The results suggest that model scale is an important factor for generalizing altered rules, though algebraic reasoning under false facts remains an open challenge even for highly advanced LLMs. This benchmark shows a possible method of measuring a model’s adaptability to faulty statements via prompts.

Future work should aim to extend FAME to encompass a broader range of mathematical concepts and other knowledge domains, exploring more diverse false fact types and integrating with direct knowledge editing algorithms. Ultimately, by exposing models to logical corruption, FAME provides a diagnostic lens to understand and measure the current LLM architectures’ adaptability to facts given via prompting methods.

ACKNOWLEDGMENTS

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

REFERENCES

A MODEL INPUT PROMPTS

The prompt instructs the model as follows:

You will be given one or more incorrect math facts. Assume every listed fact is true for this problem and use them when necessary to evaluate the question.

You may show reasoning or explanation. However, provide only the final numerical result inside `<answer></answer>` tags. Do NOT put any reasoning inside the `<answer></answer>` tags.

Formatting rules:

- The answer must be a decimal number formatted like a float, with

as many decimal places as possible.

- When a decimal appears in the problem statement, interpret it as the nearest common fraction whenever possible (e.g., $0.143 \approx 1/7$, $0.333 \approx 1/3$, $0.667 \approx 2/3$), but still output the final result as a decimal with maximal precision.
- If the problem requires solving for x , put only the numeric value of x inside the `<answer>` tags (no words, symbols, or units).
- Do not include anything other than the numeric answer(s) between the `<answer>` tags.

Example:

Incorrect facts: $2+2=5$, $1+3=2$

Question: Find the value of $((1+3)+2)*3$

(Internal reasoning, not to be shown: $((1+3)+2)*3 = (2+2)*3 = 5*3 = 15$)

`<answer>15.000000</answer>`

Below are the incorrect math facts and the question:

Incorrect facts: [insert false mathematical facts]

Question: [insert expression or algebra problem]

The prompt for control experimentation — used for ACAP/AIAP/PD calculations — instructs the model as follows:

You will be given a math problem. Assume all normal rules of mathematics are correct and use them to evaluate the question.

You may show reasoning or explanation. However, provide only the final numerical result inside `<answer></answer>` tags. Do NOT put any reasoning inside the `<answer></answer>` tags.

Formatting rules:

- The answer must be a decimal number formatted like a float, with as many decimal places as possible.
- When a decimal appears in the problem statement, interpret it as the nearest common fraction whenever possible (e.g., $0.143 \approx 1/7$, $0.333 \approx 1/3$, $0.667 \approx 2/3$), but still output the final result as a decimal with maximal precision.
- If the problem requires solving for x , put only the numeric value of x inside the `<answer>` tags (no words, symbols, or units).
- Do not include anything other than the numeric answer(s) between the `<answer>` tags.

Example:

Question: Find the value of $((1+3)+2)*3$

(Internal reasoning, not to be shown: $((1+3)+2)*3 = (4+2)*3 = 6*3 = 18$)

`<answer>18.000000</answer>`

Below is the question:

Question: [insert expression or algebra problem]